Documentación técnica consolidada — Fase 3

Climate Monitoring System
Entrega de Fase 3 — 21 de octubre de 2026

Arquitectura, funcionalidades, contratos, operación y validación del sistema.

Contenido de la entrega

- FASE 3 DOCUMENTACION CONSOLIDADA

Participación:

Christopher Herrera 100

Erwin Ramirez 100

Cinthia Robles 100

Climate Monitoring System — Documentación consolidada de Fase 3

Fecha: 21 de octubre de 2026. Este documento describe la implementación del sistema, sus funcionalidades, contratos, componentes y procedimientos de operación. Reúne frontend, backend, persistencia, comunicación y resultados de validación.

1. Visión general

El sistema registra mediciones climáticas por sensor y comunidad, simula estaciones, evalúa riesgos, administra la atención de alertas, conserva eventos, registra auditoría y genera ocho tipos de informes PDF. La interfaz Angular consume un Gateway único. Cada microservicio es propietario de sus datos y expone contratos HTTP para que otros servicios consulten su dominio.

Los componentes activos son Frontend, Gateway, Auth, Community, Sensor, Monitoring, Alert, Event, Audit y Report. Docker Compose ejecuta 17 contenedores: diez componentes de aplicación, tres PostgreSQL, dos MongoDB, SQL Server y RabbitMQ. Los servicios .NET anteriores de Identity, Alert y Audit se conservan como código de referencia; los proveedores activos de esos dominios son NestJS, Spring Boot y FastAPI respectivamente.

2. Mapa de tecnologías y microservicios

Componente	Lenguaje y framework	Persistencia	Responsabilidad	Ubicación en backend, salvo frontend
Frontend	TypeScript, Angular 22.2.3, TypeScript 6.0.3	Estado de interfaz	Pantallas, formularios, gráficos, permisos visuales y reportes	frontend/climate-monitoring-web/ src/app
Gateway	C#, ASP.NET Core .NET 10, YARP	Sin base propia	Enrutamiento, JWT, CORS, límites y WebSocket	src/Gateway/ Climate.Gateway
Auth	TypeScript, NestJS, Node.js	PostgreSQL exclusivo	Usuarios, roles, login, JWT, logout e identidad de servicio	src/Services/ AuthService
Community	C#, ASP.NET Core .NET 10	PostgreSQL exclusivo	Catálogo de comunidades, estados, ubicación y filtros	src/Services/ CommunityServic e/ Climate.Communi ties.Api
Sensor	C#, ASP.NET Core .NET 10	SensorDb en SQL Server	Sensores, tipos, unidades, ubicación y referencias a comunidades	src/Services/ SensorService
Monitoring	C#, ASP.NET Core .NET 10, SignalR	MonitoringDb en SQL Server	Lecturas, simulación, dashboard y actualización en vivo	src/Services/ MonitoringService
Alert	Java 21, Spring Boot 3.5.16, JDBC	PostgreSQL exclusivo	Reglas, evaluación de riesgos, alertas y atención	src/Services/ AlertServiceJava
Event	C#, ASP.NET Core .NET 10	EventDb en SQL Server	Historial de eventos climáticos y estadísticas	src/Services/ EventService

Audit	Python, FastAPI	MongoDB exclusivo	Consumo de auditoría, deduplicación e historial	src/Services/ AuditServicePython
Report	Python, FastAPI, Jinja2, WeasyPrint	MongoDB exclusivo y volumen PDF	Trabajos, consultas HTTP, generación, historial y descarga	src/Services/ ReportService
Mensajería	RabbitMQ	Volumen de mensajes	Entrega asíncrona de auditoría desde Outbox	Configuración Docker Compose

Las bibliotecas Python compartidas de seguridad y publicación están en src/Services/PythonCommon. El frontend utiliza Signals, componentes standalone, rutas lazy, RxJS, Chart.js, Tailwind CSS y SCSS.

3. Mapeo de rutas públicas

Ruta en Gateway	Servicio destino	Función
/api/auth y /api/users	Auth NestJS	Autenticación y administración de usuarios
/api/communities	Community .NET	Comunidades y sus estados
/api/sensors	Sensor .NET	Catálogo y administración de sensores
/api/monitoring	Monitoring .NET	Lecturas y simulación
/api/dashboard	Monitoring .NET	Resumen agregado para la interfaz
/api/alerts y /api/alert-rules	Alert Java	Alertas, atención y reglas
/api/events	Event .NET	Historial y estadísticas de fenómenos
/api/audit	Audit FastAPI	Consulta de auditoría
/api/reports	Report FastAPI	Generación, historial y descarga PDF

Gateway transforma las rutas REST públicas al prefijo interno /api/v1 de cada dominio. También transporta la conexión WebSocket de SignalR. Las APIs internas de servicio se protegen mediante clave interna y no se publican como rutas de navegador. El contrato público OpenAPI está en backend/docs/openapi/climate-api-v1.json.

4. Propiedad de datos e integración

Auth, Community y Alert usan PostgreSQL independientes; Audit y Report usan MongoDB independientes. Sensor, Monitoring y Event mantienen bases SQL Server separadas. Compartir motor no implica compartir tablas ni responsabilidad de dominio.

Community es propietario del catálogo de comunidades. Sensor valida las referencias consultando Community por HTTP y mantiene snapshots en memoria para la consulta; no crea una segunda copia persistente del catálogo. Community obtiene los conteos de sensores mediante la API interna de Sensor. La migración EF de Sensor retira la antigua relación de clave foránea local para permitir esta separación.

Report obtiene información mediante las APIs de Community, Sensor, Monitoring, Alert y Event. Su MongoDB almacena parámetros, trabajos, metadatos y descargas. Las consultas de informes no acceden directamente a las bases de otros servicios.

Los modelos intercambian UUID, fechas con zona, estados y tipos de sensor compatibles entre C#, TypeScript, Java y Python. El Gateway conserva el contrato del cliente durante el cambio de implementación de los servicios.

5. Autenticación, roles y seguridad

Auth implementa login con usuario y contraseña, administración de usuarios, roles y desactivación. La respuesta de autenticación conserva accessToken, expiresAt y user. Se admite la verificación de hashes ASP.NET Identity PBKDF2 V2/V3 para compatibilidad con usuarios transferidos.

Los JWT usan firma HS256, issuer ClimateMonitoring.Identity y audience ClimateMonitoring.Client. Los validadores comprueban firma, expiración, emisor y audiencia. Se incluyen roles y permisos explícitos junto con el claim de rol compatible con .NET. Logout registra auditoría, elimina la sesión del cliente y Auth rechaza el identificador de token cerrado en sus propios endpoints; los demás servicios aplican la expiración del JWT.

Rol	Generar reportes	Consultar historial	Descargar PDF
Administrator	Sí	Todos	Sí
Operator	Sí	Propios	No
Viewer	No	Propios	No

Los permisos reports:generate, reports:read y reports:download se verifican en backend. El navegador no elige autor, archivo físico ni URL de dependencia. Para consultar dominios, el worker de Report obtiene de Auth una identidad Viewer de servicio con vida de cinco minutos mediante un endpoint interno protegido.

Los secretos se suministran por entorno. Los archivos .env privados y las contraseñas no se incluyen en la documentación. Gateway agrega CorrelationId y aplica controles de acceso y límites de solicitudes. Los PDF usan almacenamiento privado y recursos de renderizado restringidos.

6. Comunidades y sensores

Las comunidades incluyen nombre, descripción, coordenadas, municipio, departamento, país, estado y fecha de creación. La API admite filtros, activación, desactivación y consultas paginadas. Los sensores incluyen código, nombre, descripción, tipo, unidad, comunidad y coordenadas. Se validan códigos, referencias y rangos geográficos antes de guardar.

La desactivación preserva el registro y sus referencias históricas. El catálogo activo determina los sensores procesados por la simulación. La interfaz conserva formularios, confirmaciones, mensajes de error y controles según el rol.

7. Lecturas y simulación

Monitoring registra lecturas asociadas a sensor y comunidad, con tipo, valor, unidad y fecha. Expone lecturas actuales, última lectura por sensor, historial, gráficos y búsqueda paginada por comunidad, sensor, tipo y rango temporal.

El simulador utiliza el catálogo de sensores activos y genera lotes periódicos. Se dispone de acciones para iniciar, detener y reiniciar la simulación y de valores de simulación por sensor. La configuración local utiliza un intervalo de 30 segundos. Las nuevas mediciones recorren las mismas APIs y evaluación de riesgos que las lecturas registradas manualmente.

La demostración inicial contiene Ciudad de Guatemala, Puerto Barrios y Quetzaltenango, con cinco sensores por comunidad: temperatura, humedad, velocidad de viento, lluvia y nivel de agua. Se cargan 27 reglas y primeras mediciones para sensores sin datos. La carga busca comunidades por nombre, sensores por código y reglas por nombre, conserva cambios existentes y se puede repetir sin duplicar catálogos.

8. Reglas, alertas y eventos

Alert Java administra umbrales mínimos y máximos por tipo de sensor, fenómeno y nivel. Los fenómenos son inundación, sequía, tormenta, helada e incendio forestal. Las reglas iniciales distribuyen umbrales Yellow, Orange y Red para nueve combinaciones de tipo y fenómeno.

La evaluación selecciona la mayor gravedad por fenómeno, evita duplicaciones por lectura y mantiene las transiciones de alerta activa, atendida y cerrada. Conserva responsables, fechas y snapshots de la regla que originó la alerta. PostgreSQL utiliza índices, bloqueos por fila y advisory locks para coordinar las operaciones concurrentes.

Flujo de medición: Sensor proporciona el contexto; Monitoring almacena la lectura; Alert evalúa reglas; Event registra el historial; Monitoring publica actualizaciones SignalR; los productores envían auditoría mediante Outbox y RabbitMQ. La entrega hacia Event y SignalR se registra de forma durable después del commit y admite reintentos.

9. Auditoría y mensajería

Las operaciones de negocio registran su evento Outbox en la misma transacción de persistencia. Un publicador independiente entrega el mensaje con confirmación RabbitMQ. La caída del broker o del consumidor no incorpora una llamada de auditoría remota dentro de la transacción de negocio.

Audit FastAPI consume eventos, deduplica por EventId y conserva el historial en MongoDB. El contrato incluye nombres de evento compatibles, SchemaVersion y CorrelationId. Se utilizan confirmaciones, reintentos y tratamiento de mensajes rechazados. La interfaz consulta el historial mediante paginación.

10. Dashboard y experiencia de usuario

El dashboard muestra comunidades, sensores activos e inactivos, lecturas actuales, cantidad de alertas activas y distribución por nivel. Incorpora eventos por fenómeno, evolución y estadísticas de las últimas 24 horas agrupadas por tipo y unidad: cantidad, promedio, mínimo y máximo. El filtro de comunidad se aplica a las consultas correspondientes.

SignalR actualiza la información y la recuperación REST reconstruye el estado al reconectar. La interfaz diferencia carga, ausencia de resultados, error y recuperación. Se mantienen navegación por rol, diseño móvil, foco de diálogos, recorrido de teclado, Escape, restitución de foco y validaciones asociadas a sus campos.

11. Ocho tipos de reportes PDF

Tipo	APIs de origen	Contenido
COMMUNITIES	Community, Sensor	Ubicación, estado y cantidad de sensores
SENSORS	Sensor, Community	Identificación, tipo, unidad, comunidad y estado
READINGS	Monitoring y catálogos	Mediciones con fechas, sensor, comunidad y unidades
ALERTS	Alert y catálogos	Fenómeno, nivel, valor, umbral, mensaje y atención
EVENTS	Event y catálogos	Historial de fenómenos y estados
SENSOR_STATUS	Sensor, Monitoring	Actividad, comunicación y última lectura
CLIMATE_STATISTICS	Monitoring, Alert	Cantidades, promedios, extremos y alertas del periodo
EXECUTIVE	APIs de dominio	Cobertura, indicadores, niveles y conclusiones cuantificadas

Los filtros admiten sensor, comunidad, tipo, municipio, departamento, búsqueda, actividad, fenómeno, estado y fechas según el informe. READINGS y CLIMATE_STATISTICS requieren rango temporal. EXECUTIVE utiliza las últimas 24 horas si no se especifican fechas. El rango máximo configurable es de 366 días. Las fechas incluyen zona horaria.

Las agregaciones utilizan Decimal y separan tipos y unidades. La lluvia se acumula cuando la configuración confirma mediciones incrementales en milímetros; otras semánticas se presentan sin inventar acumulados. El estado sin comunicación se calcula con un umbral predeterminado de 900 segundos, mientras que un sensor desactivado se identifica como inactivo.

12. API y ciclo de generación

Método y ruta	Respuesta
GET /api/reports/types	Catálogo de los ocho tipos
POST /api/reports/generate	HTTP 202, identificador y estado inicial del trabajo
GET /api/reports	Historial con items, total, page y pageSize
GET /api/reports/statistics	Totales y distribución por estado con control de usuario
GET /api/reports/{id}	Metadatos y progreso
GET /api/reports/{id}/download	Archivo PDF autenticado con nombre de descarga

El trabajo pasa por cola, procesamiento y finalización correcta o fallida. MongoDB conserva usuario, parámetros, fechas, progreso, error, tamaño y SHA-256. El worker reclama el trabajo atómicamente, renueva un lease de 60 segundos cada 15 segundos y recupera ejecuciones interrumpidas hasta tres intentos. La UI consulta el estado cada cinco segundos y presenta historial paginado de 25 entradas por página.

Las consultas de datos usan páginas de 250 registros. El límite predeterminado es de 50 000 filas y el exceso produce un error explícito. WeasyPrint procesa HTML Jinja2 en un subproceso con tiempo máximo de 180 segundos y tamaño máximo de PDF de 25 MiB. El diseño utiliza A4 horizontal, identificación CM, filtros, usuario, fecha, zona horaria, encabezados repetidos y numeración.

La concurrencia por usuario se protege mediante índice MongoDB único sobre el cupo activo. En la prueba de 20 solicitudes simultáneas se aceptaron tres y se respondieron 17 con HTTP 429. Los cupos se liberan al finalizar, fallar o agotar intentos.

Los archivos se identifican mediante UUID del servidor y permanecen en report-files. La retención predeterminada es de 30 días; el historial persiste después de la expiración. Descargar antes de completar responde 409 y descargar un archivo expirado responde 410. Se registra cada descarga y se auditan generación y descarga mediante publicación durable.

13. Organización del frontend

La funcionalidad de reportes se ubica en src/app/features/reports, con servicio HTTP tipado, formulario, historial, detalle y descarga. Las rutas se registran en src/app/app.routes.ts y el menú en src/app/core/layout/navigation.model.ts. Dashboard y Audit adaptan sus modelos y consultas a los contratos actuales.

La aplicación usa componentes standalone, Signals y OnPush, formularios validados y rutas lazy. La configuración /runtime-config.json define el Gateway público para REST y SignalR. El contenedor frontend sirve Angular y su configuración pública; las bases y endpoints internos se consultan desde backend.

14. Ejecución y configuración local

Desde backend:

```
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/Start-Phase3.ps1 -NoBuild
```

El comando valida Compose, inicia los servicios, espera sus healthchecks, carga la demostración y activa la simulación. Omitir -NoBuild construye las imágenes. Frontend: http://localhost:4200. Gateway: http://localhost:8080. El proyecto Docker es climate-phase3-local.

Initialize-Phase3.ps1 crea .env.phase3.local cuando no existe y preserva las credenciales existentes. El usuario inicial es admin y su contraseña se consulta en AUTH_ADMIN_SEED_PASSWORD del archivo privado. PHASE3_DEMO_ENABLED=false permite iniciar sin carga automática. Seed-Phase3Demo.ps1 completa los datos iniciales cuando los servicios ya están ejecutándose.

El Compose predeterminado contiene la arquitectura completa y los overlays docker-compose.phase3*.yml son su fuente de configuración. scripts/update-phase3-compose.ps1 regenera el archivo. Los nombres de volúmenes respetan el proyecto Docker, separando instalaciones y pruebas. Se construyeron diez imágenes de aplicación climate/*:phase3.

15. Respaldo y recuperación

Se respaldan ocho volúmenes de datos: SQL Server, RabbitMQ, tres PostgreSQL, dos MongoDB y archivos PDF. El procedimiento validado realiza snapshots con servicios detenidos, verifica SHA-256 y restaura en otro proyecto. Report MongoDB y report-files se recuperan conjuntamente para conservar la correspondencia entre metadatos y archivos.

La recuperación comprobó login, comunidades, sensores, lecturas, reglas, alertas, eventos, auditoría, historial y descarga PDF con hash idéntico. Las herramientas de transferencia SQL hacia PostgreSQL y MongoDB verifican UUID, campos, conteos y fingerprints, admiten repetición y rechazan colisiones sin sobrescribir el origen.

16. Configuración de infraestructura

Los manifiestos Kubernetes incluyen servicios, probes, recursos, secretos y almacenamiento PDF de 5 GiB con estrategia Recreate y acceso RWO. Se validaron 29 recursos y un Secret mediante esquema. Las bases se configuran como dependencias privadas. La operación con varias instancias de Monitoring requiere coordinación del simulador y distribución de SignalR mediante backplane o afinidad.

La configuración conserva interoperabilidad JWT HS256 y una tabla de identidades externas para integrar proveedores de identidad. Los archivos de infraestructura y las guías de configuración forman parte del repositorio técnico.

17. Resultados de validación

Área	Resultado registrado
.NET	122 pruebas aprobadas y una omitida
Alert Java	Siete pruebas aprobadas con integración PostgreSQL
Auth	Cinco pruebas aprobadas
Servicios Python	Once pruebas aprobadas
Angular	56 pruebas aprobadas, lint y build correctos
Accesibilidad	Seis escaneos aislados y 16 sobre Compose real sin violaciones detectadas en los recorridos evaluados
Reporte de volumen	5 000 lecturas, 20 páginas de API, PDF de 1 061 586 bytes en 38,312 segundos
Concurrencia	Tres trabajos aceptados y 17 respuestas 429 entre 20 solicitudes simultáneas
Demostración	Tres comunidades, 15 sensores, 27 reglas; lecturas de 45 a 60 y seed sin duplicados
Operación	17 contenedores saludables y recuperación

	verificada de ocho volúmenes

La integración verifica JWT, roles, filtros, generación de los ocho PDF, MIME, tamaño, hash, expiración, fallos de dependencias, recuperación de trabajos y continuidad del negocio ante fallos RabbitMQ. Los recorridos de navegador cubren login/logout, navegación, reglas, formularios, móvil, teclado y descarga.

Los resultados describen las ejecuciones realizadas y su entorno de prueba. Los escaneos de accesibilidad documentan los recorridos evaluados y las mediciones de rendimiento corresponden al equipo y datos utilizados.

18. Referencias técnicas de mantenimiento

Contratos: backend/docs/openapi/climate-api-v1.json. Pruebas: backend/docs/FASE_3_PRUEBAS.md. Evidencias: backend/docs/validation/phase3-demo.json, phase3-large-report.json, phase3-report-integration.json y phase3-recovery.json.

Scripts de validación: Test-Phase3Auth.ps1, Test-Phase3Alerts.ps1, Test-Phase3Audit.ps1, Test-Phase3System.ps1, Test-Phase3Compose.ps1, Test-Phase3Recovery.ps1 y Test-Phase3Demo.ps1. Frontend: scripts/test-browser.mjs, scripts/test-reports-browser.mjs, npm run test:ci, npm run lint y npm run build.

El generador backend/scripts/generate-phase3-word.py --consolidated-only reconstruye este Word desde la fuente Markdown consolidada. Requiere Python y python-docx. El documento reúne la arquitectura vigente y los comportamientos implementados, con referencias a código y evidencia para facilitar su exposición y mantenimiento.